



UNIVERSIDAD AUTÓNOMA METROPOLITANA
Unidad Azcapotzalco

División de Ciencias Básicas e Ingeniería
Licenciatura en Ingeniería en Computación



Manual de Instalación — Sistema CyAD

Proyecto Tecnológico

Manual de Instalación versión

Trimestre 2026-Primavera

[TU NOMBRE]

[MATRÍCULA]

al[MATRÍCULA]@azc.uam.mx

[Grado. Nombres Apellidos del Asesor]

[Profesor Titular/Asociado/Asistente]

[Departamento de Sistemas]

[correo@azc.uam.mx]

21 de agosto de 2026

Declaratoria

En caso de que el Comité de Estudios de la Licenciatura en Computación apruebe la realización de la presente propuesta, otorgamos nuestra autorización para su publicación en la página de la División de Ciencias Básicas e Ingeniería.

[TU NOMBRE]

[Grado. Nombres Apellidos del Asesor]

1. Introducción

Este manual describe cómo instalar y ejecutar el *Sistema para la gestión de formularios sobre los cursos y autoevaluaciones docentes* de la División de Ciencias y Artes para el Diseño (CyAD) en un entorno de desarrollo local. La instalación en producción sigue los mismos pasos, cambiando únicamente el archivo `.env` y el modo de servido (*gunicorn* + *Nginx*).

2. Prerrequisitos

Software	Versión mínima	Uso
Python	3.11+ (probado con 3.13)	Ejecución del backend Django.
Node.js	20+ (probado con 22 LTS)	Ejecución del frontend Vite + React.
PostgreSQL	14+ (probado con 17)	Base de datos relacional.
Git	2.40+	Clonado del repositorio.
<i>npm</i> (incluido con Node)	10+	Gestor de paquetes JS.

En Windows se recomienda utilizar *Git Bash* o *PowerShell*. En Linux/macOS los comandos son idénticos, cambiando la ruta del entorno virtual (`bin/` en lugar de `Scripts/`).

Nota importante para Windows: el instalador oficial de PostgreSQL no agrega `psql` al PATH del sistema. Si al escribir `psql -version` obtiene “command not found”, agregue manualmente la carpeta `C:\Program Files\PostgreSQL\17\bin` (ajuste al número de versión que tenga) a la variable de entorno `Path` de Windows, o invoque `psql` por ruta completa.

3. Clonado del repositorio

```
git clone <URL_del_repositorio> proyecto_terminal_cyad
cd proyecto_terminal_cyad
```

4. Configuración del backend

4.1. Entorno virtual

```
cd backend
python -m venv .venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # Linux/macOS
pip install -r requirements.txt
```

4.2. Base de datos

El proyecto utiliza por defecto el usuario postgres y una base de datos llamada proyecto_terminal_cyad (ver backend/.env.example). La forma recomendada es dejar que el script auxiliar cree la base de datos automáticamente después de configurar el archivo .env (siguiente sección):

```
python scripts/create_db.py
```

Si prefiere crearla manualmente con psql:

```
psql -U postgres
CREATE DATABASE proyecto_terminal_cyad;
```

También puede definir un usuario dedicado con permisos limitados (recomendado en despliegues compartidos) y ajustar DB_USER/DB_PASSWORD en el archivo .env en consecuencia:

```
CREATE USER cyad_user WITH PASSWORD 'cyad_pass';
GRANT ALL PRIVILEGES ON DATABASE proyecto_terminal_cyad TO
cyad_user;
```

4.3. Archivo .env

Ubicación: el archivo .env que Django lee debe estar en la carpeta backend/ (donde vive manage.py), *no* en la raíz del repositorio. El repositorio incluye backend/.env.example como plantilla; cópielo y ajuste los valores:

```
cd backend
cp .env.example .env
```

Contenido esperado (valores por defecto de la plantilla; ajuste DB_PASSWORD y SECRET_KEY a su entorno):

```
DEBUG=True
SECRET_KEY=cambia-esta-clave-por-una-aleatoria-larga
ALLOWED_HOSTS=localhost,127.0.0.1
DB_NAME=proyecto_terminal_cyad
DB_USER=postgres
DB_PASSWORD=tu_password_de_postgres
DB_HOST=localhost
DB_PORT=5432
JWT_ACCESS_MINUTES=60
JWT_REFRESH_DAYS=7
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

Nota: existe también un archivo `.env.example` en la raíz del repositorio; *no* es el que lee Django y su esquema de variables JWT (`ACCESS_TOKEN_LIFETIME`, `REFRESH_TOKEN_LIFETIME`) *no* coincide con lo que espera `backend/config/settings.py`. Utilice siempre `backend/.env.example`.

Para producción se debe establecer `DEBUG=False`, definir `ALLOWED_HOSTS` con el dominio real y generar una `SECRET_KEY` con `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`.

4.4. Migraciones

```
python manage.py migrate
```

4.5. Datos iniciales

La forma más rápida de dejar el sistema listo para usar es ejecutar el *seed* de demostración, que es idempotente y crea catálogos, UEAs, usuarios de prueba, periodo activo y el formulario de autoevaluación en un solo comando:

```
python scripts/seed_demo.py
```

De forma alternativa, se pueden cargar los catálogos base y los usuarios mínimos por separado (útil cuando ya existen datos):

```
python manage.py loaddata catalogos/fixtures/departamentos.json
python manage.py loaddata catalogos/fixtures/licenciaturas.json
python manage.py loaddata catalogos/fixtures/posgrados.json
python manage.py loaddata catalogos/fixtures/areas.json
python manage.py loaddata catalogos/fixtures/periodos.json
python scripts/seed_users.py
```

Al finalizar, existirán las siguientes cuentas:

Rol	Correo	Contraseña
Administrador	admin@cyad.uam.mx	Admin1234!
Profesor	profesor@cyad.uam.mx	Profesor1234!

Estas contraseñas deben cambiarse antes del despliegue a producción.

4.6. Servidor de desarrollo del backend

```
python manage.py runserver 8000
```

El backend queda disponible en `http://localhost:8000`. La documentación interactiva de la API está en `http://localhost:8000/api/docs/`.

5. Configuración del frontend

En una terminal aparte:

```
cd frontend
npm install
npm run dev
```

El frontend queda disponible en `http://localhost:5173`. Vite realiza *hot reload* automático al editar archivos fuente.

Si el backend no está en `localhost:8000`, cree `frontend/.env.local` con:

```
VITE_API_URL=http://tu-servidor:8000/api/v1
```

Nota: el nombre correcto de la variable es `VITE_API_URL` (verificable en `frontend/src/api/client.js`).

6. Verificación

1. Abra `http://localhost:5173/` — debe mostrarse la vista pública “Explorar”.
2. Inicie sesión con `admin@cyad.uam.mx`. Debe redirigir al *dashboard* de administrador.
3. Cierre sesión, inicie con `profesor@cyad.uam.mx`. Debe redirigir al *dashboard* de profesor.
4. Ejecute la suite de pruebas: `pytest` desde `backend/`. Todas deben pasar.

7. Comandos administrativos útiles

Comando	Uso
<code>python manage.py cargar_catalogos_csv <archivo></code>	Importar catálogos desde CSV.
<code>python manage.py createsuperuser</code>	Crear cuenta administradora adicional.
<code>python manage.py dumpdata <app></code>	Exportar datos a JSON.
<code>pytest -cov=. -cov-report=html</code>	Ejecutar pruebas con cobertura.
<code>npm run build (frontend)</code>	Compilar producción estática (dist/).

8. Solución de problemas

- **Error FATAL: password authentication failed:** verifique `DB_USER` y `DB_PASSWORD` en `.env` y que el usuario tenga permisos sobre la base de datos.
- **Error CORS al iniciar sesión desde el frontend:** verifique que `CORS_ALLOWED_ORIGINS` incluya `http://localhost:5173`.
- **429 Too Many Requests al iniciar sesión:** se activó el *rate limiting* (5 intentos por minuto). Espere 60 segundos y reintente.
- **Puerto 5173 ocupado:** Vite usará automáticamente el siguiente puerto libre; verifique la URL impresa al inicio.
- **seed_demo.py falla con "La suma de pesos de las secciones debe ser 100 %"**: en algunas versiones del script las secciones se crean sin peso, lo que hace fallar la publicación del formulario. Como *workaround* temporal, edite `backend/scripts/seed_demo.py` y añada `peso=Decimal("50")` a la creación de cada una de las dos `Seccion.objects.create(...)`, luego ejecute `python manage.py flush --noinput` y vuelva a lanzar el seed. Ver `hallazgos_despliegue.pdf` para detalles.
- **psql: command not found en Windows:** agregue `C:\Program Files\PostgreSQL\<ver>` al PATH, o llame a `psql` por ruta completa. Alternativamente, use *pgAdmin* o el script `scripts/create_db.py`, que utiliza el driver Python y no requiere `psql` en el PATH.